

partAtlas

Status	Concept
--------	---------

Stack	Python
-------	--------

Repo Name	partAtlas
-----------	-----------

Entwicklung eines lokalen, Linux-nativen Digital-Twin-Katalogs ([partAtlas](#)), der große 3D-Druck-Sammlungen (5.000+ Modelle) performant erschließt, bitgenauen G-Code für reproduzierbare 1-Klick-LAN-Drucke ohne Cloud-Zwang persistiert und Spulenbestände sowie Sensor-Telemetrie via [flatgraphdb](#) und [flatTSDB](#) lückenlos auditiert.

- **Projekt: partAtlas (3D Print Digital Twin & Asset Navigation)**
 - **Document:** User Requirement Specification & Architektur-Konzept
 - **Version:** 1.5.0
 - **Status:** Entwurf / Pilot-Spezifikation (Konsolidiert mit 3MF-Katalog UI-Mustern, User-Feedback & Multi-Printer-Roadmap)
 - **Produktlinie:** Additive-Manufacturing-Vertikale im Produkt-Ökosystem
(flatgraphdb + flatTSDB + eliteVault + graphAtlas)
 - **Basis-Technologie:** flatgraphdb (Graph-Engine) + flatTSDB (Time-Series Engine)
 - **Zielgruppe:** Additive Fertigung, Prototypenbau, Open-Source-Maker und Linux-Power-User mit hohen Qualitäts-, Wiederholbarkeits- und Performanceansprüchen
 - **Referenz-Benchmark & UX-Fundament:** Übernahme und Härtung der bewährten UI- und Organisationskonzepte aus dem Open-Source-Projekt *3MF Katalog Manager* (Bexxs75)

• 1. Ausgangslage & Problemstellung

1.1 Voice of Customer (Original User-Zitate / O-Töne)

Die folgenden Praxiseindrücke markieren die zentralen Schmerzpunkte und Leitplanken für das Projekt:

“Wenn man den Drucker direkt damit verbinden könnte, könnte man sich sogar die Cloud-Anbindung der Druckerhersteller sparen.”

“Man kann das Objekt im Slicer öffnen lassen, ich glaube auch direkt drucken. Das habe ich aber noch nicht probiert, weil der Katalog bei mir (wahrscheinlich wegen der alten Hardware) nicht gescheit funktioniert.”

“Mein Ordner ‘3D-Druck’ hat inzwischen auch schon 5.700 Einträge. Wenn ich da was suche, finde ich es nicht oder nur sehr zeitaufwändig. Da geht es mit dem Katalog (sofern er funktioniert) schon 100x schneller.”

“Bei mir funktioniert der 3MF Katalog nur teilweise. Bei 90% der Dateien macht er keine Bilder ODER Verknüpfungen.”

“Es müsste so sein, dass man einen funktionierenden G-Code 1:1 an den Drucker senden kann aus der App und das 3D-Modell genauso in 3 Monaten drucken kann. So müsste es sein.”

1.2 Das Problem mit traditionellem 3MF & bestehenden Katalogen

Das **.3mf**-Format bündelt Geometrie, XML-Metadaten und Slicer-Profile in einem ZIP-Container. In Verbindung mit herkömmlichen Desktop-Katalogen führt dies in der Praxis zu folgenden Engpässen:

1. **Fragiler Ingest & Thumbnail-Verlust (Das 90%-Problem):** Viele 3D-Dateien (reine STLs, STEP, ältere oder mit Fremdslicern exportierte 3MFs) bringen keine gerenderten PNG-Thumbnail mit. Bisherige Kataloge scheitern daran, bleiben hängen oder zeigen leere Platzhalter.
2. **Skalierungs- und Hardware-Grenzen (5.700+ Dateien):** Bestehende Werkzeuge basieren oft auf unoptimierten UI-Rendern oder tragen Standard-Datenbanken. Bei Sammlungen von 5.000 bis 10.000+ Modellen friert die UI ein, und Suchen dauern quälend lange.
3. **Fehlende Wiederholbarkeit (Repetierbarkeit):** Ein erneutes Slicen nach Updates von OrcaSlicer oder Bambu Studio verändert oft Schichtpfade oder Kühlzeiten. Ein verlässlicher Nachdruck gelingt nur mit dem bitgenauen, unveränderten **G-Code**.
4. **Keine direkte Maschinensteuerung (Cloud-Zwang):** Viele Tools bieten nur das Öffnen im Slicer oder rein lesende Klipper-Endpunkte. Für den eigentlichen Druck zwingen Hersteller die Anwender häufig in proprietäre Clouds, statt direkt und lokal per LAN zu übertragen.
5. **Entkoppeltes Spulen- & Verbrauchs-Tracking:** Das Filamentlager existiert getrennt von der tatsächlichen Ausführung an der Maschine.

6. **Mangelnde Flotten-Koordination (Multi-Printer-Hürde):** Werden mehrere Drucker parallel betrieben (z. B. Voron für ABS/Nylon, Bambu für Mehrfarb-Deko, Bettschubser für Prototypen), fehlt die logische Verknüpfung: Welcher Drucker ist frei, welcher hat das passende Material geladen und welcher Maschinencode passt auf welches Druckbett?

1.3 Die Zielvision von partAtlas

Ein nativer, robuster Digital-Twin-Katalog auf Basis des **Unified Stacks**

(`flatgraphdb` + `flatTSDB`):

- **Vertraute, geliebte UX:** 1:1-Übernahme der ergonomischen Muster des *3MF Katalogs* (Hashtag-Bar, AMS-Slot-Visualisierung, Bounding-Box-Karten, Warteschlange).
- **Ausfallsicherer 100%-Thumbnail-Ingest:** Automatischer Headless-Background-Renderer für alle Modelle ohne eingebettetes Vorschaubild.
- **Extreme Performance & Minimal Footprint:** Sub-Sekunden-Suche über 10.000+ Modelle hinweg, Virtual Scrolling und `< 80 MB` RAM-Nutzung im Leerlauf auf Linux-Systemen (inkl. Raspberry Pi / Klipper-Hosts).
- **Cloudless Direct Printing:** 1:1 Übertragung des validierten G-Codes direkt per LAN an Klipper (Moonraker) oder Bambu Lab (LAN-Modus / MQTT).
- **Multi-Printer-Transparenz:** Parallele Übersicht über alle lokalen Drucker, deren Live-Status und aktuelle AMS-/Spulenbelegungen.
- **Auditfähige Historie:** Lückenlose Aufzeichnung von Temperatur- und Sensorverläufen in `flatTSDB` synchron zum Druckauftrag.

• 2. Benutzer-Persona & UX-Leitplanken

2.1 Persona: Der pragmatische Linux- & Maker-Power-User

- **Hardware-Realität:** Nutzt ältere Laptops, Refurbished-PCs oder Raspberry Pis (Klipper-Hosts) als Steuerzentrale. Verlangt Software, die nicht überhitzt und ohne Speicherlecks läuft.
- **Bestandsdaten:** Verwaltet gewachsene Archive von 5.000 bis über 10.000 Dateien (STL, 3MF, STEP, G-Code) in tief verschachtelten Verzeichnissen.
- **Toolchain & Maschinenpark:** Betreibt oft 2 bis 5 verschiedene Drucker (z. B. Klipper-Voron/RatRig, Bambu Lab P1S/X1C, Prusa MK4), nutzt OrcaSlicer / PrusaSlicer (bevorzugt als AppImage unter Linux) und meidet Hersteller-Clouds.
- **Werte:** Absolute Datensouveränität (Local-First, kein Cloud-Zwang), blitzschnelle Orientierung und verlässliche Reproduzierbarkeit von Drucken.

2.2 Detaillierte UX-Spezifikation (Orientiert an 3MF Katalog)

Das Interface übernimmt die Kernstärken des *3MF Katalogs* und ergänzt sie um Power-User-Aktionen:

1. Hashtag-Schnellfilter-Leiste (Direkt unter der Suchleiste):

- Automatisch generierte und manuell vergebene Tags als klickbare Chips: `#Funktional`, `#Deko`, `#Haushalt`, `#mehrteilig`, `#mehrfarbig`, `#Vasenmodus`, `#miniatur`.
- Sofortiges Filtern der Trefferliste mit einem Klick ohne modale Dialoge.

2. Erweiterte Modellkarten-Anatomie (Grid-Kachel):

- **Visual:** Generiertes oder extrahiertes 3D-Thumbnail.
- **Dateiformat-Badge:** Farblich codiert (`[.3mf]`, `[.stl]`, `[.step]`, `[.obj]`).
- **Status- & Verbrauchs-Badge:** z. B. `[✓ Gedruckt · 27.04 g]` oder `[Nur Geometrie]`.
- **Bauraum-Abmessungen (Bounding Box):** Sofortanzeige der Dimensionen (z. B. $52 \times 60 \times 122$ mm), um Bett-Kompatibilität direkt zu prüfen.
- **Gewicht & Material:** Geschätzter oder gesLICETER Verbrauch (z. B. 48.2 g \ `vert` PLA).

3. Drucker- & AMS/MMU-Slot-Manager:

- Haptische Visualisierung der Druckerhardware mit ihren Materialfächern:
 - Bambu AMS / AMS lite (Slots 1–4)
 - Prusa MMU3 (Slots 1–5)
 - Creality CFS / Anycubic ACE Pro
 - Klassische Einzel-Spulenhalter
- Drag-and-Drop von Filament-Spulen aus dem Lager direkt in die virtuellen Slots.

4. „Reicht das Filament?“-Plausibilitätsprüfung:

- Ampelsystem im Inspector: Vergleicht den Soll-Verbrauch der aktiven Druckplatte mit dem Restgewicht der im Ziel-Slot geladenen Spule.

5. Druckwarteschlange & Protokoll:

- **Warteschlange (Queue):** Anstehende Drucke einreihen und per Drag & Drop priorisieren.

- **Interaktives Protokoll:** Liste aller vergangenen Drucke; Klick öffnet das Telemetrie-Audit.

• 3. Funktionale Anforderungen (Functional Requirements)

3.1 Modell- & Asset-Management mit Robustem Ingest (FR-01)

- **FR-01.1 (Multi-Format-Ingest & Archiv-Entpackung):**

- Unterstützung für `.3mf`, `.stl`, `.step`, `.obj`.
- Direktes Einlesen und Entpacken gepackter Archive (`.zip`, `.7z`, `.rar`, `.tar`), ohne das Dateisystem zu fragmentieren.

- **FR-01.2 (Resilientes Thumbnail-Handling & Headless-Fallback):**

- *Schritt 1:* Versuch, embedded Thumbnails aus der `.3mf` (ZIP) zu extrahieren.
- *Schritt 2 (Fallback für STLs & ältere 3MFs):* Fehlt ein Thumbnail, rendert ein integrierter, schlanker Background-Worker (z. B. via Software-Rasterizer/EGL oder Open CASCADE für STEP) automatisch orthogonale und isometrische Ansichten.
- *Garantie:* **100% visuelle Abdeckung – keine leeren Kacheln mehr.**

- **FR-01.3 (Deduplizierung & Bounding Box):**

- SHA-256-Hashing über die rohen Mesh-Geometrien zur Identifikation identischer Modelle.
- Automatische Vermessung der Modellgrenzen ($X \times Y \times Z$ in mm) beim Datei-Scan.

- **FR-01.4 (Baugruppen- & Multi-Part-Klammerung):**

- Logische Gruppierung mehrteiliger Baugruppen über Container-Knoten im Graphen.

3.2 G-Code-Management & Cloudless 1-Click Re-Print (FR-02)

- **FR-02.1 (Maschinencode als persistentes Asset):**

- Validierter G-Code wird als eigenständiges Artefakt unveränderlich abgelegt und mit Modell, Druckerprofil und Material verknüpft.

- **FR-02.2 (Slicer-Header-Parsing):**

- Extraktion von Druckzeit, Schichthöhe, Infill-Muster, Nozzle-Temperatur und exaktem Filament-Verbrauch (Gramm/Volumen) direkt aus den G-Code-Kommentaren (Bambu Studio, OrcaSlicer, PrusaSlicer).

- **FR-02.3 (Cloud-Free Direct LAN Printing):**

- Direkte Übertragung des G-Codes per 1-Klick über das lokale Netzwerk:

- **Klipper / Moonraker:** REST-Upload und Druckstart-Trigger via WebSocket.
- **Bambu Lab LAN-Modus:** Lokale Übertragung via FTPS und Ausführungstrigger via lokalem MQTT (vollständige Umgehung der Bambu Cloud).
- **OctoPrint / PrusaLink:** Lokale REST-API-Übergabe.
- **FR-02.4 (Slicer-Fallback):**
 - Schnell taste “Im Slicer öffnen” ruft installierte Slicer lokal auf (z. B. OrcaSlicer Applimage via Linux CLI-Call `orcaslicer %f`).

3.3 Spulenlager & AMS/MMU-Slot-Tracking (FR-03)

- **FR-03.1 (Filament-Lager):**
 - Erfassung physischer Spulen (Hersteller, Material, Farbe als HEX-Code, Chargen-Nr., Restgewicht, Preis pro kg für Kostenberechnung).
- **FR-03.2 (AMS/MMU-Slot-Mapping):**
 - Virtuelle Zuweisung von Spulen an konkrete Drucker-Slots (z. B. *Bambu AMS Slot 2: PETG Schwarz*).
- **FR-03.3 („Reicht das Filament?“-Ampel):**
 - Pre-Flight-Prüfung:
$$\text{Restgewicht}(\text{Slot-Spule}) \geq \text{Soll-Verbrauch}(\text{G-Code-Platte})$$

Visuelle Signalisierung: Grün (ausreichend), Gelb (knapp, < 10% Reserve), Rot (Filament-Abriss droht).
- **FR-03.4 (Automatisches Verbrauchs-Tracking):**
 - Nach Bestätigung oder via MQTT-Statusmeldung nach Druckende wird der Ist-Verbrauch automatisch vom Spulenkonto abgebucht.

3.4 Telemetrie- & Prozessdaten-Historie via flatTSDB (FR-04)

- **FR-04.1 (Lokaler Sensor-Stream):**
 - Kontinuierliches Abgreifen der Sensordaten während des Drucks über Klipper WebSocket oder Bambu LAN MQTT (sekündliche Abtastung).
- **FR-04.2 (Sekundengenaue Speicherung):**
 - Speicherung von Ist- und Soll-Temperaturen (Hotend, Bed), Lüftergeschwindigkeiten und Schichtfortschritt in `flatTSDB`.
- **FR-04.3 (Qualitäts-Audit & Soll/Ist-Vergleich):**

- Interaktive Verknüpfung im Druckprotokoll: Synchroner Plot von Solltemperatur (aus G-Code) und Isttemperatur (aus flatTSDB) zur Fehleranalyse bei Fehldrucken.

3.5 High-Scale-Suche & Performance (FR-05)

- **FR-05.1 (Instant-Filter für 10.000+ Modelle):**
 - Volltext-, Attribut- und Tag-Suche mit Sub-Sekunden-Antwortzeit (< 150 ms) über 10.000+ Modelle via In-Memory-Graphstrukturen.
- **FR-05.2 (Virtual Scrolling Grid):**
 - Das Katalog-Grid rendert ausschließlich Kacheln im aktuellen Viewport, um DOM-Größe und RAM-Bedarf auf alten Systemen minimal zu halten.

3.6 Multi-Printer-Flottenmanagement & Smart Dispatching (FR-06)

- **FR-06.1 (Paralleles Multi-Drucker-Monitoring):**
 - Gleichzeitige Überwachung beliebiger lokaler Druckerinstanzen im Mischbetrieb (z. B. 2x Klipper/Moonraker, 1x Bambu P1S via LAN, 1x Prusa MK4 via PrusaLink).
 - Statusanzeige im Sidebar-Panel: *Bereit (Idle)*, *Druckt (mit Fortschritt in %)*, *Pausiert*, *Offline*.
- **FR-06.2 (Smart Matching & Eignungsprüfung):**
 - Wählt der Anwender ein Modell oder G-Code-Artefakt aus, prüft das System automatisch die Eignung der verfügbaren Drucker:
 - **Bauraum-Validierung:** Passt die Bounding Box des Modells auf das Druckbett der jeweiligen Maschine?
 - **Nozzle- & Profil-Prüfung:** Stimmen Nozzle-Durchmesser (z. B. 0.4 mm vs. 0.6 mm) und Kinematik überein?
 - **Material- & Slot-Verfügbarkeit:** Hat der Ziel-Drucker das passende Filament in ausreichender Menge im AMS/Spulenhalter geladen?
- **FR-06.3 (Farm-Warteschlange & Job-Dispatching):**
 - Warteschlangen-Aufträge können manuell oder regelbasiert an den nächsten freien, kompatiblen Drucker übergeben werden.

• 4. Nicht-funktionale Anforderungen (NFR)

- **NFR-01 (Linux-Native & Minimal Footprint):**

- Native Lauffähigkeit auf Standard-Linux-Distributionen (Debian, Ubuntu, Fedora, Arch) sowie auf ARM64 (Raspberry Pi 4/5).
- Bereitstellung als Standalone-Binary, Systemd-Daemon oder schlankes Appliance.
- **NFR-02 (Low-Spec Hardware-Eignung):**
 - RAM-Nutzung im Leerlauf $< 80 \text{ MB}$.
 - Verzicht auf schwerfällige Framework-Monolithen (keine speicherhungrige Standard-Electron-App; stattdessen optimierte Web-Engine / Tauri oder schlankes Python-Backend mit optimiertem UI-Layer).
- **NFR-03 (Local-First / Zero Cloud Dependency):**
 - Sämtliche Daten (Graph, Zeitreihen, Binär-Assets, Render-Bilder) verbleiben im lokalen Dateisystem oder Heimnetzwerk.
 - Vollständige Funktionalität ohne Internetverbindung.
- **NFR-04 (Robuste Fehlertoleranz):**
 - Absturzsicherer Ingest: Schlägt das Parsen einer defekten 3D-Datei fehl, wird diese übersprungen und geloggt; der Katalog bleibt zu 100% stabil.
- **5. Datenmodell & Graph-Architektur (flatgraphdb + flatTSDB)**

5.1 Knoten-Typen (Node Types)

Text

```
[MODEL_ASSET]      -> Logische Identität des Modells (z.
B. "Quadrocopter_Arm")
[PART_GEOMETRY]    -> Rohe Mesh-Geometrie (STL/STEP-Hash,
Bounding Box X/Y/Z)
[GCODE_ARTIFACT]   -> Maschinencode (1:1 wiederholbar,
herstellerspezifisch)
[PRINT_PROFILE]    -> Slicer-Soll-Vorgaben (0.20mm, 4 Wände,
40% Gyroid)
[MATERIAL_SPOOL]   -> Physische Spule im Lager (ID: #9041,
PETG, 350g Rest)
[MATERIAL_MASTER]  -> Abstrakter Filamenttyp (z. B. "Prusament
PETG Jet Black")
[PRINTER_DEVICE]   -> Lokaler Drucker (z. B. "Voron 2.4",
"Bambu P1S", "RatRig V-Core")
[AMS_SLOT]         -> Konkretes Ladefach am Drucker (z.
B. "AMS_1_Slot_2")
[PRINT_JOB]        -> Protokollierter Drucklauf (Audit-Trail)
[TAG_ITEM]         -> Klickbare Hashtags (z. B.
"#Funktional", "#Deko")
```

5.2 Kanten-Taxonomie

Text

```
(MODEL_ASSET)      —[HAS_PART]—————> (PART_GEOMETRY)
(MODEL_ASSET)      —[HAS_GCODE]—————> (GCODE_ARTIFACT)
(MODEL_ASSET)      —[HAS_TAG]—————> (TAG_ITEM)

(GCODE_ARTIFACT)   —[BASED_ON_PROFILE]—————> (PRINT_PROFILE)
(GCODE_ARTIFACT)   —[REQUIRES_MATERIAL]—————> (MATERIAL_MASTER)
(GCODE_ARTIFACT)   —[COMPILED_FOR]—————> (PRINTER_DEVICE)

(PRINTER_DEVICE)   —[HAS_SLOT]—————> (AMS_SLOT)
(AMS_SLOT)         —[LOADED_WITH]—————> (MATERIAL_SPOOL)
(MATERIAL_SPOOL)   —[IS_TYPE_OF]—————> (MATERIAL_MASTER)

(PRINT_JOB)        —[EXECUTED_GCODE]—————> (GCODE_ARTIFACT)
(PRINT_JOB)        —[EXECUTED_ON]—————> (PRINTER_DEVICE)
(PRINT_JOB)        —[USED_SPOOL]—————> (MATERIAL_SPOOL)
```

5.3 Anbindung an flatTSDB

Der Knoten `PRINT_JOB` verknüpft das statische Modell mit den Live-Sensordaten:

```
{ } JSON

{
  "node_id": "job_2026_09_30_0842",
  "type": "PRINT_JOB",
  "properties": {
    "status": "COMPLETED",
    "duration_seconds": 8540,
    "printer_target": "Klipper_Voron_LAN",
    "cloud_bypass": true,
    "gcode_file": "storage/gcode/arm_front_v1.gcode",
    "tsdb_stream_id": "printer_voron_stream_8841",
    "tsdb_time_range": [1790750400, 1790758940]
  }
}
```

- **6. UI/UX-Spezifikation (Detaillierte Oberflächen-Architektur)**

```

|
|
| partAtlas [Linux / Local-First / flatgraphdb]
| 🔍 Suche in 5.742... |
|
|
| HASHTAGS: [#Alle] [#Funktional] [#Deko] [#Haushalt]
| [#mehrteilig] [#mehrfarbig] [#miniatur]|
|
|
| FILTER & FLOTTE | MODELL-GRID (Virtual Scrolling)
| INSPECTOR / DETAIL PANEL |
|
|
| ▼ Drucker-Flotte |
|
| [ Thumb 1 ] [ Th 2 ] [ Th 3 ] |
| • Voron 2.4 | [.3mf] [✓ Gedruckt] | [.stl]
| [Auto] | (3-Perspektiven-Vorschau) |
| (Klipper LAN | |
|
| Bereit) | (3D Render Preview) | (Render-
Fallb) | Modell: Arm_Front.3mf |
| • Bambu P1S | |
| Bounding Box: 52×60×122 mm |
| (Druckt 42%) | Arm_Front | Base_Plate
| Tags: #Funktional #Drohne |
| o Prusa MK4 | 52 × 60 × 122 mm | 120×120×15
mm |
| (Offline) | 42.4 g | PETG | 88.0 g | PLA
| Ziel-Drucker & Match-Status: |
|
| [• Voron 2.4 (Klipper LAN) ▼] |
| ▼ Formate |
| ✓ Bauraum OK (Bett: 300×300) |
| ■ .3mf (3.2k) | [.3mf] [In Queue] | [.step]
| [CAD] | ✓ Material geladen (Slot 1) |
| ■ .stl (2.1k) | Motor_Mount | Gear_Shaft
|
| □ .step (442) | 80 × 40 × 30 mm | 20×20×85 mm
| „Reicht das Filament?“-Ampel: |
| | 18.5 g | ABS | (Nur CAD)
| Bedarf: 42.4g | Vorrat: 350g |
| ▼ Warteschlange |
| [====|-----] GRÜN (OK) |
| 1. Arm_Front | Zeige 1-24 von 5.742 Modellen
|

```

	2. Top_Plate	
	[▶ 1-Click LAN Re-Print]	
	(Direkt via Klipper LAN)	
	▼ Sammlungen	
	[↗ Im Slicer öffnen]	
	▶ Drohne V2	
	(OrcaSlicer AppImage)	
	▶ Voron Mods	
	[📊 Telemetrie & Audit]	

- **7. End-to-End Workflow (Ingest bis Druck)**

Text

```
[ Ordner mit 5.700 Dateien (3MF, STL, STEP, G-Code, ZIP) ]
|
▼
[ Ingest-Service (Linux) ]
├─ 1. ZIP- / Header-Prüfung:
│   └─ Thumbnail vorhanden? —> Extrahiere
│       embedded PNG
│       └─ FEHLT? (STL / Alt) —> Headless-Background-
│           Renderer generiert 3 Ansichten
│       └─ 2. Mesh-Analyse: Berechne Bounding Box (X/Y/Z) & SHA-
│           256 Mesh-Hash
│       └─ 3. G-Code-Header-Parsing: Extrahiere exakten
│           Filamentbedarf & Druckdauer
│       |
│       ▼
│       [ Update flatgraphdb ]
│       └─ Knoten & Kanten idempotent aktualisieren
│           (Keine Duplikate)
│       |
│       ▼
│       [ Druck-Vorbereitung (UI) ]
│       └─ Nutzer wählt Modell im Grid
│       └─ Filter via Hashtag-Bar (`#Funktional`)
│       └─ Flotten-Check: Welcher Drucker ist frei und
│           kompatibel (Bauraum, Nozzle, AMS)?
│       └─ Ampel-Prüfung: Prüfe Spule in AMS-Slot gegen
│           Modell-Verbrauch
│       |
│       ▼
│       [ Druck-Ausführung (Cloudless) ]
│       └─ Klick auf [ ► 1-Click LAN Re-Print ]
│       └─ Übertragung: 1:1 G-Code via LAN an ausgewählte
│           Zielmaschine (Klipper / Bambu LAN)
│       └─ Startbefehl via WebSocket / LAN-MQTT
│       └─ flatTSDB: Sekündliche Aufzeichnung von Düsen-, Bett-
│           und Vibrationsdaten
│       └─ Nach Druckende: Automatischer Abzug der Grammanzahl
│           von der Spule
```

• 8. Phasenplan für das Pilotprojekt

• Phase 1: Performanter Linux-Katalog & Resilienter Ingest (4 Wochen)

- High-Speed-Indexierung bestehender Sammlungen (5.700+ Dateien).

- Headless-Renderer für STLs/STEPS zur Beseitigung des 90%-Thumbnail-Problems.
- Virtuelles Grid mit Hashtag-Bar, Bounding-Box-Karten und 3-Perspektiven-Vorschau.
- **Phase 2: Cloudless Direct Print & G-Code-Historie (3 Wochen)**
 - Speicherung des bitgenauen G-Codes als unveränderliches Artefakt.
 - Direkter 1-Klick-Druck via LAN an Klipper (Moonraker) und Bambu Lab (LAN-Modus) ohne Hersteller-Cloud.
 - Direkte Anbindung an OrcaSlicer / PrusaSlicer (Applmage unter Linux).
- **Phase 3: AMS-Slots, Verbrauchs-Tracking & flatTSDB-Audit (3 Wochen)**
 - AMS/MMU-Slot-Manager mit virtueller Spulenzuweisung.
 - „Reicht das Filament?“-Prüfampel mit automatischem Spulenabzug.
 - flatTSDB -Kopplung zur sekundlichen Temperatur- und Fehleraufzeichnung synchron zum Druckauftrag.
- **Phase 4: Multi-Printer-Flotte & Smart Dispatching (Weiterführender Schritt - 3 Wochen)**
 - Paralleles Multi-Device-Monitoring (Klipper-, Bambu- und Prusa-Flotten im Mischbetrieb).
 - Smart Matching: Automatisches Vorschlagen des optimalen freien Druckers anhand von Bauraum, Nozzle und AMS-Spulenbelegung.
 - Flottenweites Warteschlangen-Management (Farm-Modus).
- **Codezeilen-Abschätzung & Modul-Dimensionierung (partAtlas)**
 - **Projekt:** partAtlas (3D Print Digital Twin & Multi-Printer Catalog)
 - **Technologie-Basis:** Python Backend (FastAPI / Native Daemon) + Leichtgewichtiges UI (Tauri / Svelte / HTML5 Canvas) auf flatgraphdb und flatTSDB
 - **Metrik:** Physische Source Lines of Code (SLOC, ohne Leerzeilen und reine Kommentarblöcke, mit Type Hints und Dokumentation)

1. Architektur- & Komplexitäts-Annahmen

1. **Kein Framework-Ballast:** Verzicht auf monolithische Enterprise-Frameworks. Nutzung fokussierter Libraries (z. B. polars / pydantic für Ingest, trimesh / moderngl für Headless-Rendering, paho-mqtt & httpx für Drucker-APIs).

2. **Engines als Bibliotheken:** `flatgraphdb` und `flatTSDB` werden als bestehende Python-Module/C-Bindings importiert und sind **nicht** Teil dieser Zählung (sie zählen als vorgelagerte Plattform-Engines).
3. **Produktionsstandard:** Saubere Modularisierung, Pydantic-Schemas für Datensicherheit, asynchrone I/O-Routinen (`asyncio`) und ca. 20-25% Testabdeckung (Unit- & Integrationstests).

2. Modul-Aufschlüsselung je Phase

Phase 1: High-Speed Katalog & Resilienter Ingest (4 Wochen)

Fokus: Dateisystem-Indexierung, Hash-Prüfung, Headless-Thumbnail-Generierung, virtuelles Grid und Hashtag-Suche.

- **Backend / Core Ingest:**

- File-Watcher & Archiv-Entpacker (`zip` , `7z` , `tar` Handling): ~450 SLOC
- Mesh-Parser & Geometrie-Vermessung (Bounding Box, SHA-256): ~400 SLOC
- Headless-Renderer (EGL / Off-Screen Software-Rasterizer für STL/STEP): ~650 SLOC
- `flatgraphdb` -Ingest-Mapper & Cache-Layer: ~500 SLOC
- Lokale REST/WebSocket-API (FastAPI/IPC): ~450 SLOC

- **Frontend (Desktop UI):**

- Virtual Scrolling Grid (performante Kachel-Wiederverwendung für 10k+ Items): ~750 SLOC
- Hashtag-Filterbar & Schnellsuche-State: ~400 SLOC
- Modellkarten-Komponenten & 3-Perspektiven-Vorschau: ~600 SLOC

- **Tests & Fixtures:** ~850 SLOC

Delta Phase 1: ~5.050 SLOC **Kumulativ Phase 1: ~5.050 SLOC**

Phase 2: Cloudless Direct Print & G-Code-Historie (3 Wochen)

Fokus: Slicer-Header-Parsing, G-Code-Persistenz, LAN-Übertragung (Klipper Moonraker / Bambu MQTT & FTPS), ApplImage-Integration.

- **Backend / Print Execution:**

- Slicer-Header-Parser (OrcaSlicer, PrusaSlicer, Bambu Studio Metadaten): ~380 SLOC
- G-Code-Asset-Manager (Deduplizierung, Integritäts-Check): ~320 SLOC

- Klipper/Moonraker LAN Client (REST Upload + WS Trigger): ~550 SLOC
- Bambu Lab LAN Client (Lokales FTPS-Push + Lokales MQTT-Command-Interface): ~650 SLOC
- Slicer-CLI-Launcher (Linux Desktop Exec Handler / ApplImage-Runner): ~180 SLOC
- **Frontend:**
 - 1-Click LAN Print Dialog & Target-Selector: ~450 SLOC
 - G-Code-Inspektor & Header-Detailansicht: ~400 SLOC
- **Tests & Mocks (Virtuelle Drucker-Endpunkte):** ~750 SLOC

Delta Phase 2: ~3.680 SLOC **Kumulativ Phase 2: ~8.730 SLOC**

Phase 3: Spulenlager, AMS/MMU & flatTSDB-Audit (3 Wochen)

Fokus: Spulenverwaltung, Fach-Visualisierung, „Reicht das Filament?“-Ampel, Sensor-Stream-Subscriber und interaktive Telemetrie-Kurven.

- **Backend / Warehouse & Telemetrie:**
 - Filament-Lager & Spulen-Inventar (CRUD + Chargen-Tracking): ~450 SLOC
 - AMS/MMU-Slot-Mapping & automatischer Spulen-Abzug nach Druck: ~400 SLOC
 - Telemetrie-Bridge: MQTT/Klipper-Stream-Subscriber → flatTSDB -Append: ~550 SLOC
 - Audit-Service (Soll-Ist-Abgleich G-Code vs. TSDB-Sensorkurve): ~380 SLOC
- **Frontend:**
 - Haptischer AMS/MMU-Slot-Visualisierer (Drag & Drop Spulenfächer): ~650 SLOC
 - „Reicht das Filament?“-Ampelindikator im Inspector: ~250 SLOC
 - Interaktiver Telemetrie-Plot (Canvas2D/WebGL-Chart: Soll- vs. Ist-Temperatur): ~700 SLOC
- **Tests & Fixtures:** ~650 SLOC

Delta Phase 3: ~4.030 SLOC **Kumulativ Phase 3: ~12.760 SLOC**

Phase 4: Multi-Printer-Flotte & Smart Dispatching (3 Wochen)

Fokus: Paralleles Monitoring heterogener Drucker, Kompatibilitäts-Matrix (Bauraum/Nozzle/Material), Farm-Warteschlange.

- **Backend / Fleet Engine:**
 - Flotten-Status-Aggregator (paralleles Polling & Event-Dispatching): ~550 SLOC
 - Smart-Matching-Algorithmus (Bauraum-Validierung, Nozzle, Slot-Prüfung): ~480 SLOC
 - Farm-Warteschlangen-Logik (Job-Priorisierung, Re-Routing): ~420 SLOC
- **Frontend:**
 - Multi-Drucker-Sidebar mit Live-Status & Fortschrittsbalken: ~550 SLOC
 - Flottenweites Queue-Management (Drag & Drop Re-Ordering): ~450 SLOC
 - Drucker-Match-Statusanzeige im Inspector: ~300 SLOC
- **Tests & Flotten-Simulation:** ~600 SLOC

Delta Phase 4: ~3.350 SLOC **Kumulativ Phase 4: ~16.110 SLOC**

3. Kumulierte Übersichtstabelle

Phase	Kern-Lieferumfang	Delta (SLOC)	Kumulativ (SLOC)	Aufwand (Wochen)
Phase 1	Katalog, Ingest, Headless-Render, Virtual Grid	~5.050	~5.050	4
Phase 2	G-Code-Storage, Klipper/Bambu LAN Direct-Print	~3.680	~8.730	3
Phase 3	Spulen/AMS-Manager, flatTSDB-Stream & Audit-UI	~4.030	~12.760	3
Phase 4	Flotten-Monitoring, Smart Dispatching, Farm-Queue	~3.350	~16.110	3

4. Code-Verteilung nach Schichten (Gesamtumfang ~16.000 SLOC)

- **Backend-Logik & Hardware-Protokolle (Python):** ca. **52%** (~8.400 SLOC)

- *Schwerpunkte:* MQTT/FTPS-Treiber, Headless-Rendering, Slicer-Parsing, Graph/TSDB-Anbindung.
- **Frontend & Interaktionsschicht (Svelte/JS/Canvas):** ca. **30%** (~4.850 SLOC)
 - *Schwerpunkte:* Virtual Grid, AMS-Slots, Telemetrie-Plot, Flotten-Status.
- **Automatisierte Tests & Mocks:** ca. **18%** (~2.850 SLOC)
 - *Schwerpunkte:* Mocks für Klipper/Bambu, synthetische 3MF/G-Code-Generatoren, Graph-Integritätstests.

5. Einordnung & Machbarkeit

- **Kompakter Footprint:** Ein Gesamtumfang von rund **16.000 SLOC** ist für eine vollwertige Desktop- und Steuerzentrale dieser Leistungsfähigkeit sehr schlank (zum Vergleich: ausgewachsene Electron-Monolithen liegen oft jenseits von 60.000–100.000 SLOC).
- **Entwickler-Geschwindigkeit:** Durch die Auslagerung der Kernkomplexität in `flatgraphdb` (Graph-Logik) und `flatTSDB` (Zeitreihen) spart das Projekt schätzungsweise 8.000 bis 12.000 Zeilen eigenen Datenbank- und Indexierungs-Code ein.
- **Ressourceneffizienz:** Dieser schlanke Codebestand gewährleistet, dass das Binary schnell kompiliert, innerhalb von Sekundenbruchteilen startet und auf älterer Hardware problemlos mit unter 80 MB Arbeitsspeicher auskommt.